TP555 - AI/ML

Lista de Exercícios #5

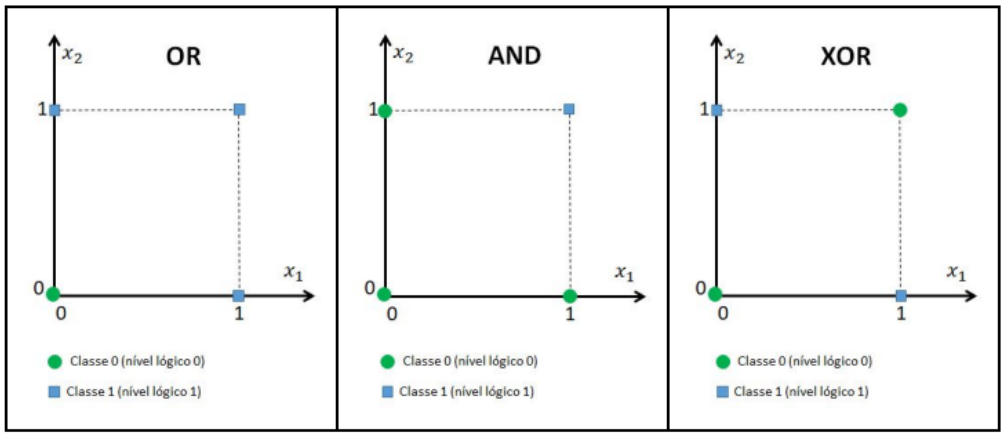
Classificação Linear: Parte 2

1. Dado as funções lógicas: OR, AND e XOR, representadas pelas tabelas e gráficos abaixo. Responda quais podem ser classificados com um classificador linear, ou seja, uma linha reta, que separa as duas classes. Caso algum deles não possa ser separado linearmente, que tipo de classificador seria necessário?

OR		
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	1

AND		
x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

XOR		
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0



2. Suponha que você tenha um problema de classificação com múltiplas classes, ou seja, $Q > 2$, (então $y \in \{1, 2, \ldots, Q\}$). Usando o método um-contra-todos, quantos classificadores de regressão logística diferentes você precisaria treinar para realizar a classificação destas classes?

3. Suponha que você deseje prever, a partir dos atributos x de um tumor, se ele é maligno ($y = 1$) ou benigno ($y = 0$). Um classificador de regressão logística gera, para um tumor específico, $h_a(x) = P(y = 1|x; a) = 0,7$, portanto estima-se que haja 70% de chance de esse tumor ser maligno. Qual seria a estimativa para $P(y = 0|x; a)$, ou seja, a probabilidade de o tumor ser benigno?

4. Considere a regressão logística com 2 atributos, x_1 e x_2. Suponha que $a_0 = 5$, $a_1 = -1$ e $a_2 = 0$, de tal forma que $h_a(x) = f(5-x_1)$. Encontre e desenhe a fronteira de decisão. Mostre as regiões em que o classificador classifica $y = 1$ (classe positiva) e $y = 0$ (classe negativa).

5. Suponha que você quisesse classificar fotos como externas/internas e diurnas/noturnas. Nesse caso, você deve implementar dois classificadores de regressão logística ou um classificador de regressão Softmax?

6. Utilizando a base de dados `20 Newsgroups` da biblioteca `scikit-learn`, classifique os textos do conjunto de validação (ou testes) em uma das 4 categorias: `talk.religion.misc`, `soc.religion.christian`, `sci.space` ou `comp.graphics`. Apresente a matriz de confusão e as principais métricas de classificação utilizando a função `classification_report` (veja a dica abaixo).

   - **Dica**: Veja como baixar e usar a base de dados no exemplo: ClassifyingTextMultinomialNB.ipynb.
   - **Dica**: Você só precisa baixar as 4 categorias de texto e dividi-las em 2 conjuntos: um de treinamento e outro de validação, conforme feito no exemplo acima.
   - **Dica**: Documentação da função `classification_report`: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html.

7. Utilizando a seguinte função hipótese $h_a(x) = f(a_0 + a_1x_1 + a_2x_2)$, onde $f(.)$ é a função de limiar sigmoide (ou logística), e o algoritmo do gradiente descendente em batelada com *early-stopping*, treine um classificador linear para classificar os dados gerados através da execução do código abaixo.

```
# Import all necessary libraries.
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_circles

# Create concentric circles.
x, y = make_circles(n_samples=1000, random_state=42, noise=0.1,
    factor=0.2)

# Split array into random train and test subsets.
x_train, x_test, y_train, y_test = train_test_split(x, y,
    random_state=23)
```

   Em seguida, faça o seguinte:

   a. Plote um gráfico mostrando as diferentes classes.
   b. Analisando o gráfico do item (a), que tipo de fronteira de decisão seria necessária para separar essas classes (linear ou não-linear)?
   c. Plote um gráfico com número de épocas versus os erros de treinamento e validação.
   d. Plote a matriz de confusão.
   e. Plote uma figura com as fronteiras de decisão.
   f. Plote o gráfico com a curva característica de operação do receptor (ROC).
   g. Imprima as métricas de classificação utilizando a função `classification_report`
   h. Repita os itens (c) até (g), agora com a seguinte função hipótese: $h_a(x) = f(a_0 + a_1x_1 + a_2x_2 + a_3x_1^2 + a_4x_2^2)$

i. Qual a diferença na performance do classificador entre as duas funções hipóteses? (**Dica:** qual das 2 hipóteses confere ao classificador maior precisão de classificação?)

(**Dica:** utilize como base o notebook ClassificationOfTwoLinearlySeparableClasses.ipynb, nele você vai encontrar a implementação do algoritmo do gradiente descendente em batelada com early-stopping).

(**Dica:** não se esqueça de encontrar o melhor valor para o passo de aprendizagem).

8. Neste exercício, você irá implementar o gradiente descendente em batelada com *early-stopping* para a regressão Softmax (ou seja, para classificação quando $Q > 2$) sem usar a biblioteca `scikit-learn` para classificar os dados pertencentes a 3 classes diferentes. Utilize um número máximo de iterações igual a 2000, por exemplo. Utilize o trecho de código abaixo para gerar os dados das 3 classes. Esse trecho também plota uma figura mostrando os dados pertencentes às 3 classes.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score,
    classification_report
from sklearn.datasets import make_blobs
import seaborn as sns

# make 3-class dataset for classification
centers = [[-5, 0], [0, 1.5], [5, -1]]
x, y = make_blobs(n_samples=1000, centers=centers, random_state=42)

idx0 = np.argwhere(y == 0)
idx1 = np.argwhere(y == 1)
idx2 = np.argwhere(y == 2)
fig = plt.figure(figsize=(5,5))
plt.plot(x[idx0,0], x[idx0,1], '.', label='Class 0')
plt.plot(x[idx1,0], x[idx1,1], 'rx', label='Class 1')
plt.plot(x[idx2,0], x[idx2,1], 'ko', label='Class 2')
plt.xlabel('$x_1$')
plt.ylabel('$x_2$')
plt.legend()
plt.show()
```

Você pode utilizar, se desejar, as seguintes funções auxiliares.

```
def to_one_hot(y):
    n_classes = y.max() + 1
    m = len(y)
    Y_one_hot = np.zeros((m, n_classes))
    Y_one_hot[np.arange(m), y] = 1
    return Y_one_hot
def softmax(logits):
    exps = np.exp(logits)
    exp_sums = np.sum(exps, axis=1, keepdims=True)
    return exps / exp_sums
# epsilon e um valor muito pequeno, e.g. , 1e-7, usado para se
    evitar que quando y_prob for zero o resultado nao seja infinito
```

```
    epsilon igual a um valor pequeno evita esse tipo de Warning:
    "RuntimeWarning: divide by zero encountered in log"
def error_function(x, a, y, epsilon=1e-7):
    logits = x.dot(a)
    y_prob = softmax(logits)
    error = -np.mean(np.sum(y * np.log(y_prob + epsilon), axis=1))
    return error

def classifier(x, a):
    logits = x.dot(a)
    y_prob = softmax(logits)
    c = np.zeros((len(y_prob), 1))
    for i in range(0, len(y_prob)):
        c[i, 0] = np.argwhere(y_prob[i,:] == y_prob[i,:].max())[0]
    return c

def predict_prob(x, a):
    logits = x.dot(a)
    y_prob = softmax(logits)
    h1 = y_prob
    h0 = 1 - h1
    h = np.c_[h0, h1]
    return h
```

Utilizando-se a seguinte função hipótese $h_a(x) = f(a_0 + a_1x + a_2x_2)$, pede-se

a. Divida o conjunto de dados em 75% para treinamento e 25% para validação.

b. Plote um gráfico com número de épocas versus os erros de treinamento e validação.

c. Plote a matriz de confusão.

d. Plote uma figura mostrando as fronteiras de decisão.

e. Imprima as métricas de classificação utilizando a função `classification_report`.

(**Dica:** utilize como base o notebook ClassificationOfTwoLinearlySeparableClasses.ipynb, nele você vai encontrar a implementação do algoritmo do gradiente descendente em batelada com early-stopping).

(**Dica:** não se esqueça de encontrar o melhor valor para o passo de aprendizagem).

(**Dica:** lembre-se que os vetores de rótulos dos conjunto de treinamento e validação, y_train e y_test, devem ser convertidos para a representação one-hot encoding, veja a função to_one_hot definida acima).

9. Neste exercício você irá utilizar uma implementação do regressor logístico e as abordagens Um-Contra-o-Resto e Um-Contra-Um para realizar classificação multiclasses. Inicialmente, implemente uma função que realize o treinamento de um regressor logístico com *early-stopping*. Utilize no mínimo 40000 épocas para o treinamento de cada um dos classificadores. Encontre um passo de aprendizagem que seja ótimo para todos os classificadores treinados. Utilize o trecho de código abaixo para gerar exemplos pertencentes a 5 classes.

```
# Number of examples.
M = 1000
# Create 3-class dataset for classification.
centers = [[-5, 0], [0, 1.5], [5, -1], [10, 1.5], [15, 0]]
x, y = make_blobs(n_samples=M, centers=centers, random_state=seed)
```

Em seguida, faça o seguinte:

a. Implemente a abordagem para a classificação multiclasse chamada Um-Contra-o-Resto.

b. Utilizando a abordagem Um-Contra-o-Resto faça a predição utilizando toda a base de dados.

c. Gere a matriz de confusão e calcule o `accuracy_score` dessa abordagem.

d. Implemente a abordagem para a classificação multiclasse chamada Um-Contra-Um.

e. Utilizando a abordagem Um-Contra-Um faça a predição utilizando toda a base de dados.

f. Gere a matriz de confusão e calcule o `accuracy_score` dessa abordagem.

g. Qual das duas abordagens apresenta melhor resultado?

10. Neste exercício você usará sua implementação do regressor logístico utilizando o algoritmo do gradiente descendente em batelada com *early-stopping* para realizar a classificação entre 2 modulações digitais. A base de dados com os sinais *InPhase* e *Quadrature* (I e Q), além do *label* indicando a modulação estão no arquivo modulations.csv. Baixe o arquivo e o utilize para treinamento e validação do seu classificador. Baixe o arquivo e o utilize para treinamento e validação do seu classificador. Utilize o trecho de código abaixo para ler as informações do arquivo, elas estão da seguinte forma: primeira coluna amostras InPhase (I), segunda coluna amostras em Quadrature (Q) e a terceira coluna label a modulação, onde 0 indica BPSK e 1 QPSK.

```
def readCSVFile(filename):
    df = pd.read_csv(filename, header=None)
    i = df[0].to_numpy()
    q = df[1].to_numpy()
    mod = df[2].to_numpy()
    i = i.reshape(len(i),1)
    q = q.reshape(len(q),1)
    mod = mod.reshape(len(mod),)
    return i, q, mod
filename = './modulations.csv'
I, Q, mod = readCSVFile(filename)
y = I + 1j*Q
```

De posse das informações da base de dados, faça o seguinte:

a. Plote um gráfico com as amostras I e Q indicando a qual modulação cada par de amostras I e Q pertence.

b. Analisando o gráfico do item (a), que tipo de fronteira de decisão seria necessária para separar essas classes (linear ou não-linear)?

c. Treine seu regressor logístico com early stopping com pelo menos 30000 épocas. Não se esqueça de encontrar o melhor passo de aprendizagem. **OBS.:** você irá precisar encontrar uma função hipótese que consiga separar as duas classes. Analise o gráfico da letra (A) para identificar que tipo de função hipótese é necessária.

d. Plote o gráfico mostrando os erros de treinamento e validação versus o número de épocas.

e. Imprima o valor da acurácia do seu modelo, use a função `accuracy_score`.

f. Plote a matriz de confusão.

g. Plote uma figura mostrando as fronteiras de decisão.

h. Plote o gráfico com a curva característica de operação do receptor (ROC).

i. Imprima as **métricas de classificação** utilizando a função `classification_report`.